JIRA BACKLOG

Project #22 - Agile Backlog - Team 28 - CODEVERSE

1. Backlog Overview

This deliverable is aligned to Project #22, “AI-Powered Upskilling Recommendation and Automation System for Students.” The mini-project implementation is intentionally scoped to a student-facing web application: account/authentication, profile, skills, ML-based career recommendation, skill-gap analysis, learning roadmap and progress tracking. External job-portal automation, resume parsing, GitHub/LinkedIn integration, notifications and an admin portal are outside the current mini-project scope.

2. Team Information

Team	Team Name	Member SRNs
28	CODEVERSE	PES2UG24AM184, PES2UG24AM217, PES2UG24AM210, PES2UG24AM214

3. Agile Structure

The backlog uses Epics → User Stories → implementation tasks/bugs. Each story represents a user-visible or system-level increment. UI work, database work and testing are supporting implementation tasks rather than being treated as standalone functional ownership. Each team member owns a complete functional area, supporting the course requirement for individual contribution.

4. Epic Backlog

Epic	Name	Owner	Priority	Goal
EPIC-01	User Account & Profile	PES2UG24AM184	High	Enable students to create accounts and maintain profile information used by the recommendation workflow.
EPIC-02	Skill Management	PES2UG24AM217	High	Capture and maintain student skills and proficiency levels.
EPIC-03	Career Recommendation	PES2UG24AM217	High	Generate ranked career paths using the student's profile and skills.
EPIC-04	Skill Gap Analysis	PES2UG24AM210	High	Compare current skills against the requirements of a selected career.
EPIC-05	Learning Roadmap	PES2UG24AM210	High	Turn skill gaps into an ordered learning path with resources/projects/certification-oriented steps.
EPIC-06	Progress Tracking	PES2UG24AM214	High	Track completion/progress and use performance information for recommendation refresh.
EPIC-07	Platform Quality & Delivery	PES2UG24AM214	Medium	Integrate modules and satisfy project-level engineering, testing and delivery practices.

5. User Story Backlog

Epic	Story	Summary	Priority	SP	Owner SRN	Acceptance Criteria
EPIC-01	US-01	Register a student account	High	3	PES2UG24AM184	Valid registration creates one account; duplicate email is rejected; password policy is enforced.
EPIC-01	US-02	Login and logout securely	High	3	PES2UG24AM184	Valid credentials create a session; invalid credentials are rejected; logout invalidates protected access.
EPIC-01	US-03	View and edit student profile	High	5	PES2UG24AM184	Student can view and update profile fields; saved values persist after relogin.
EPIC-01	US-04	Protect authenticated routes	High	3	PES2UG24AM184	Unauthenticated requests to protected pages/APIs are denied or redirected; users cannot access another user's profile.
EPIC-02	US-05	Add skills with proficiency	High	5	PES2UG24AM217	Student can add a supported skill and proficiency level; invalid proficiency is rejected.
EPIC-02	US-06	Edit and delete skills	High	3	PES2UG24AM217	Student can update or remove an existing skill; database reflects the change.
EPIC-02	US-07	Prevent duplicate/invalid skills	Medium	3	PES2UG24AM217	Duplicate skills are handled consistently and unknown/invalid skill values do not corrupt the profile.
EPIC-02	US-08	Expose normalized skill data	Medium	3	PES2UG24AM217	Skills sent to recommendation/gap modules use a consistent normalized representation.
EPIC-03	US-09	Generate career recommendations	High	8	PES2UG24AM217	Given a valid profile and skills, the system returns ranked career paths.
EPIC-03	US-10	Show recommendation explanations	High	5	PES2UG24AM217	Each recommendation shows a concise reason or contributing skills/features where supported.
EPIC-03	US-11	Handle recommendation failures	Medium	3	PES2UG24AM217	Unavailable/failed ML service returns a controlled user-facing error without crashing the application.
EPIC-03	US-12	Version recommendation model/results	Medium	3	PES2UG24AM217	Recommendation requests/results can be associated with model version and timestamp for traceability.
EPIC-04	US-13	Select a target career	High	3	PES2UG24AM210	Student can choose one supported career path for skill-gap analysis.
EPIC-04	US-14	Calculate missing skills	High	8	PES2UG24AM210	System correctly identifies required skills not present at the target proficiency.
EPIC-04	US-15	Normalize skill comparison	Medium	5	PES2UG24AM210	Equivalent skill names are matched using the project's normalization rules.
EPIC-04	US-16	Refresh gap after skill changes	High	5	PES2UG24AM210	Adding/removing/updating skills changes the displayed gap without stale results.
EPIC-05	US-17	Generate learning roadmap	High	8	PES2UG24AM210	A valid skill gap produces an ordered roadmap containing learning objectives.
EPIC-05	US-18	Order prerequisite skills	High	5	PES2UG24AM210	Prerequisites appear before dependent skills in generated roadmaps.
EPIC-05	US-19	Attach curated learning resources	High	5	PES2UG24AM210	Roadmap steps can reference valid resource/project/certification-oriented entries.
EPIC-05	US-20	Regenerate roadmap after changes	Medium	5	PES2UG24AM210	Updated skills regenerate a roadmap without obsolete completed/irrelevant steps where applicable.
EPIC-06	US-21	Track roadmap item progress	High	5	PES2UG24AM214	Student can set progress/completion for a roadmap item within valid bounds.
EPIC-06	US-22	Display overall progress	High	5	PES2UG24AM214	Dashboard shows item-level and overall completion consistent with stored records.
EPIC-06	US-23	Persist progress across sessions	High	3	PES2UG24AM214	Progress remains correct after refresh/logout/login.
EPIC-06	US-24	Use progress for recommendation refresh	Medium	5	PES2UG24AM214	Current progress/performance data is passed to the recommendation refresh workflow.
EPIC-07	US-25	Integrate frontend and backend APIs	High	5	PES2UG24AM214	Core user journeys complete successfully through real API calls.
EPIC-07	US-26	Implement automated unit/integration tests	High	5	PES2UG24AM214	Core modules have automated tests and results are captured in CI.
EPIC-07	US-27	Configure CI/CD and quality checks	Medium	5	PES2UG24AM214	Repository pipeline runs tests and configured quality checks on changes.
EPIC-07	US-28	Prepare release/deployment package	Medium	3	PES2UG24AM214	A reproducible deployment/run procedure exists and project can be demonstrated from a clean environment.

6. Suggested Implementation Tasks

Story Group	Implementation Tasks	Definition of Done
US-01–04	Frontend forms/pages; auth API; password hashing/session handling; validation; protected routes; unit tests; PR review.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-05–08	Skill UI; skill validation/normalization; skill CRUD APIs; database persistence; API tests; PR review.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-09–12	Feature preparation; model loading/inference service; recommendation API; ranking/explanation response; model version logging; tests.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-13–16	Career selection UI/API; skill normalization; gap calculation engine; persistence/recalculation; tests.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-17–20	Roadmap generator; prerequisite ordering; resource catalog; roadmap APIs/UI; regeneration logic; tests.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-21–24	Progress APIs; completion rules; dashboard calculations; recommendation-refresh integration; tests.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.
US-25–28	API integration; end-to-end flow; CI pipeline; static/quality checks; container/run scripts; release documentation.	Code merged via PR; relevant tests pass; acceptance criteria verified; defects recorded; Jira status and traceability updated.

7. Sprint Plan

Sprint	Focus	Target Stories	Primary Owners
Sprint 1	Foundation	US-01 to US-04	PES2UG24AM184 + all for shared setup
Sprint 2	Skills	US-05 to US-08	PES2UG24AM217
Sprint 3	Recommendation	US-09 to US-12	PES2UG24AM217 + integration support
Sprint 4	Skill Gap + Roadmap	US-13 to US-20	PES2UG24AM210
Sprint 5	Progress + Integration	US-21 to US-25	PES2UG24AM214 + all
Sprint 6	Testing, CI/CD, stabilization	US-26 to US-28 + defects	All members

8. Priority and Story Point Summary

Total planned story points: 127. High-priority work is implemented first because it forms the core demonstrable product path. Story points are relative estimates and should be re-estimated by the team after the first sprint.

9. Definition of Ready

A story has a clear user/system outcome, acceptance criteria, owner, priority and dependencies identified. The team understands the data/API impact before moving it into an active sprint.

10. Definition of Done

Implementation is complete, code is reviewed through a GitHub PR, relevant unit/integration tests are added, manual acceptance is verified, defects are recorded, and the Jira story is linked to the corresponding requirement/test evidence.

11. Team Ownership

Member SRN	Primary Full Functionality	Supporting Responsibilities
PES2UG24AM184	Authentication + Student Profile	API integration, security checks, review
PES2UG24AM217	Skill Management + Career Recommendation	ML pipeline/model integration, data validation, review
PES2UG24AM210	Skill Gap Analysis + Learning Roadmap	Recommendation inputs/outputs, data model, review
PES2UG24AM214	Progress Tracking + Platform Integration	CI/CD, system testing coordination, integration

12. Jira Configuration Suggested

Create one Scrum project. Use Epics for the seven groups above; Stories for US-01–US-28; Tasks/Sub-tasks for implementation work; Bug for defects found during testing. Recommended fields: Priority, Story Points, Assignee, Sprint, Labels, Acceptance Criteria and links to GitHub PRs. Test cases should use IDs such as UT_AUTH_01 so defects can be traced back to the Project Test Plan.

13. Traceability Expectations

For each completed story, maintain the chain: SRS requirement → Jira story → GitHub branch/PR → implementation → test case → test evidence. This supports the course emphasis on Agile development, requirements traceability, code reviews and individual contribution.